AI Course Project Report

Simple Delivery Allocation — Load Balancing

Topic: Comparative & Hybrid AI | BFS Search + Genetic Algorithm

Language: Python | Libraries: NumPy, random, math, matplotlib

Team Members

Member	Section
Tarek Hossam Mohamed	3
Tarek Ahmed Ali	3
Hazem Ayman Saleh	2

0. Introduction & Problem Statement

This project addresses the **Simple Delivery Allocation (Load Balancing)** problem. A company operates from a single Depot and must deliver packages to **6 delivery points** (P1–P6) on a 2D grid containing walls. Two vehicles are available, and the objective is to distribute the 6 packages between the two vehicles such that both travel approximately the same total distance — achieving **load balance**.

Two AI approaches are compared: (1) a **BFS search algorithm** to compute the exact shortest path cost from the Depot to each delivery point, and (2) a **Genetic Algorithm (GA)** to optimize the binary assignment of packages to vehicles, minimizing workload imbalance.

0.1. Environment Design

2.1 Grid Specification

The environment is a **15 × 15 NumPy array.** Cell value 0 denotes a walkable path; cell value 1 denotes a solid wall. The grid was designed with sufficient wall density to make the routing non-trivial, forcing BFS to navigate around obstacles and ensuring the package distances are meaningfully different.

Element	Position (row, col)	Description
Depot (D)	(0, 0)	Starting point for both vehicles
P1	(2, 4)	Delivery point 1
P2	(5, 13)	Delivery point 2
P3	(7, 11)	Delivery point 3
P4	(10, 3)	Delivery point 4
P5	(12, 5)	Delivery point 5
P6	(14, 12)	Delivery point 6 — farthest from depot

Discussion Q1: Search Algorithm — Why BFS?

> **Question:** What type of search algorithm did you use and why? Compare BFS vs DFS.

1.1 Algorithm Used: Breadth-First Search (BFS)

We used **Breadth-First Search (BFS)** to compute the shortest path from the Depot to each of the 6 delivery points on the 15×15 grid. BFS is an uninformed (blind) search algorithm that uses a **FIFO queue** and expands nodes level by level — all nodes at distance 1, then distance 2, and so on.

1.2 BFS Implementation

```
def bfs(grid, start, goal):
    queue   = deque([(start, [start])])   # FIFO
    visited = {start}
    while queue:
        (r, c), path = queue.popleft()    # closest node first
        if (r, c) == goal:
            return len(path) - 1, path    # steps = len - 1
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:  # 4-directions
            if in_bounds and walkable and not visited:
                visited.add((nr, nc))
                queue.append(((nr,nc), path + [(nr,nc)]))
    return inf, []   # unreachable
```

1.3 BFS vs DFS — Full Comparison

Feature	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO)
Traversal Order	Level by level (width first)	Deep branch first
Finds Shortest Path?	YES — guaranteed optimal	NO — may find long paths first
Complete?	YES — finds solution if exists	NO — can loop infinitely in cyclic graphs
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V) — stores entire frontier	O(depth) — lower memory
Best Use Case	Shortest path in unweighted graphs	Exploring all paths, DFS spanning trees
Used in This Project?	YES	NO — would give wrong distances

1.4 Why BFS and Not DFS for This Project

We need the **exact shortest distance** from the Depot to each delivery point because these distances feed directly into the GA fitness function. A wrong distance would cause the GA to make incorrect allocation decisions.

- **BFS guarantees the shortest path** — the first time BFS reaches the goal, it has taken the minimum steps. DFS has no such guarantee.
- **Our grid has walls** — paths are not straight lines. DFS might find a long path around many walls before finding the direct route. BFS finds the correct minimum automatically.
- **BFS is complete** — it will always find a path if one exists, which is essential since an unreachable delivery point would break the entire project.

> **Key insight:** BFS is the correct and only appropriate choice for this sub-problem. DFS optimises for memory, not path length, and would undermine the accuracy of the GA fitness function.

1.5 BFS Results

Package	Delivery Point	BFS Distance	Notes
P1	(2, 4)	6 steps	Closest to Depot
P2	(5, 13)	18 steps	Far right side
P3	(7, 11)	18 steps	Mid-right area
P4	(10, 3)	17 steps	Mid-left area
P5	(12, 5)	17 steps	Lower-left zone
P6	(14, 12)	26 steps	Farthest from Depot

Discussion Q2: Fitness Function

> **Question:** What is the fitness function you are using and why?

2.1 Chromosome Structure

A chromosome is a **binary array of 6 genes**. Each gene corresponds to one delivery package. Gene = 0 means the package goes to Vehicle 1; Gene = 1 means Vehicle 2.

```
Example chromosome: [1, 0, 1, 0, 0, 1]
  Gene 0 (P1) = 1  →  Vehicle 2
  Gene 1 (P2) = 0  →  Vehicle 1
  Gene 2 (P3) = 1  →  Vehicle 2
  Gene 3 (P4) = 0  →  Vehicle 1
  Gene 4 (P5) = 0  →  Vehicle 1
  Gene 5 (P6) = 1  →  Vehicle 2
```

2.2 Fitness Formula

```
dist_V1 = sum of BFS distances for all packages where gene == 0
dist_V2 = sum of BFS distances for all packages where gene == 1

fitness(chromosome) = 1 / (1 + |dist_V1 - dist_V2|)
```

2.3 Why This Formula?

- **Objective mapping:** Our goal is to minimise |dist_V1 - dist_V2| (the workload imbalance). GAs maximise, so we need to transform a minimisation objective into a maximisation score.
- **Bounded range:** The formula always returns a value in (0, 1]. This makes fitness scores easy to compare and compatible with tournament selection.
- **Intuitive extremes:** |diff| = 0 → fitness = 1.0 (perfect balance, maximum). As imbalance grows, fitness shrinks toward 0 smoothly.
- **Alternative rejected:** fitness = −|diff| is mathematically valid but produces negative scores, which complicates roulette-wheel selection and makes results harder to interpret.

2.4 Worked Example

Best chromosome found: `[1, 0, 1, 0, 0, 1]`

Vehicle	Packages (gene value)	Distances	Total
Vehicle 1 (genes = 0)	P2, P4, P5	18 + 17 + 17	52 steps
Vehicle 2 (genes = 1)	P1, P3, P6	6 + 18 + 26	50 steps

```
fitness = 1 / (1 + |52 − 50|)
        = 1 / (1 + 2)
        = 1 / 3
        ≈ 0.3333
```

Imbalance = only **2 steps** out of 102 total — approximately 2% imbalance. This is a near-perfectly balanced allocation.

Discussion Q3: Selection Algorithm

> **Question:** What type of selection algorithm did you use and why was it the best choice for your problem?

3.1 Selection Used: Tournament Selection (k = 5)

We used **Tournament Selection** with a tournament size of k = 5. In each tournament, 5 chromosomes are randomly sampled from the population, and the one with the highest fitness is chosen as a parent. This is repeated to obtain the second parent.

```
def tournament_selection(population, distances):
    candidates = random.sample(population, TOURNAMENT_SIZE)  # k=5
    return max(candidates, key=lambda c: fitness(c, distances))
```

3.2 Why Tournament and Not Roulette Wheel?

Criterion	Tournament Selection (k=5)	Roulette Wheel Selection
Selection pressure	Controlled by k	Proportional to fitness values
Scale sensitivity	Immune to fitness scale	Breaks if all fitnesses are similar
Diversity	Good — weaker chromosomes still sometimes win	Poor — dominant chromosomes take over quickly
Premature convergence risk	Low at k=5	High when one chromosome is much better
Suitable for this project?	YES	RISKY — our fitness values span a narrow range

- **Scale immunity:** Our fitness values all fall in (0, 1]. Roulette wheel divides by the sum of all fitness scores, which can produce unstable probabilities when values are clustered near the same number. Tournament selection is unaffected by scale.
- **Tunable pressure:** k = 2 gives weak selection pressure (more exploration). k = 10 gives very strong pressure (faster but risks premature convergence). k = 5 is the standard balance for a 6-gene, 64-solution space.
- **Small search space:** With only 2^6 = 64 possible chromosomes, preserving diversity is critical. Tournament selection ensures even weaker solutions sometimes survive, preventing the population from collapsing to one solution too early.

Discussion Q4: Crossover Type

> **Question:** What type of crossover did you use and why was it the best choice?

4.1 Crossover Used: One-Point Crossover

We used **One-Point Crossover**. A random cut point cp is chosen in the range [1, 5]. The two offspring inherit the prefix from one parent and the suffix from the other.

```
def one_point_crossover(parent1, parent2):
    cp = random.randint(1, NUM_PACKAGES - 1)    # cp in {1,2,3,4,5}
    offspring1 = parent1[:cp] + parent2[cp:]
    offspring2 = parent2[:cp] + parent1[cp:]
    return offspring1, offspring2
```

Example with cp = 3:

```
Parent 1:    [1, 0, 1 | 0, 0, 1]
Parent 2:    [0, 1, 0 | 1, 1, 0]
             ———————— cp=3 ————————
Offspring 1: [1, 0, 1 | 1, 1, 0]   (P1 prefix + P2 suffix)
Offspring 2: [0, 1, 0 | 0, 0, 1]   (P2 prefix + P1 suffix)
```

4.2 Why One-Point and Not Two-Point or Uniform?

Criterion	One-Point	Two-Point	Uniform
Building block preservation	High	Medium	Low — destroys all blocks
Disruption level	Low	Medium	High
Best for chromosome length	Short (≤10 genes)	Medium (10–50 genes)	Long (50+ genes)
Suitable for 6-gene chromosome?	YES	Overkill	Too destructive

- **Short chromosome:** With only 6 genes, we want a low level of disruption. Two-point crossover swaps an inner segment — for a 6-gene chromosome this can swap 4 of the 6 genes at once, which is too aggressive.
- **Building blocks:** One-point preserves the left or right half of each parent intact. If parent A has an optimal first half and parent B has an optimal second half, one-point crossover can combine them perfectly.
- **Uniform rejected:** Uniform crossover treats each gene independently — it breaks all positional structure, which is unnecessary and counterproductive for a short binary chromosome.

Discussion Q5: Mutation Type

> **Question:** What type of mutation did you use and why was it the best choice?

5.1 Role of Mutation

Mutation is the **diversity engine** of the GA. Crossover can only recombine gene values that already exist in the population. If the optimal chromosome requires a gene combination that never appeared in generation 0, crossover alone can never find it. Mutation introduces genuinely new genetic material, allowing the GA to escape local optima and explore the full search space.

Property	Crossover	Mutation
Role	Exploit existing good solutions	Explore new solutions
Mechanism	Recombine parent genes	Randomly alter individual genes
Risk without it	Population converges to best known	Population gets stuck in local optimum
Rate	Applied to every pair	Applied with probability MUTATION_RATE = 0.3

5.2 Three Mutation Types Implemented

5.2.1 Random Mutation (Primary)

Definition: Selects one random gene index and flips its value (0 → 1 or 1 → 0).

```
def random_mutation(chromosome):
    idx          = random.randint(0, NUM_PACKAGES - 1)
    mutated      = chromosome[:]
    mutated[idx] = 1 - mutated[idx]   # binary flip
    return mutated

Example:  [1, 0, 1, 0, 0, 1]
          flip gene at index 2
Result:   [1, 0, 0, 0, 0, 1]   (P3 reassigned from V2 to V1)
```

Why best for this project: Our chromosome is binary — each gene has only two valid values (0 or 1). Random mutation on a binary chromosome is a bit-flip, which is the most natural and efficient mutation. One flip = one package reassignment. The granularity perfectly matches the problem: we want to try reassigning one package at a time, not scramble the whole assignment.

5.2.2 Swap Mutation (Inorder Mutation)

Definition: Selects two random gene indices and swaps their values.

```
def swap_mutation(chromosome):
    i = random.randint(0, NUM_PACKAGES - 1)
    j = random.randint(0, NUM_PACKAGES - 1)
    mutated    = chromosome[:]
    mutated[i], mutated[j] = mutated[j], mutated[i]
    return mutated
```

```
Example:  [1, 0, 1, 0, 0, 1]  swap indices 1 and 5
Result:   [1, 1, 1, 0, 0, 0]   (P2 and P6 swap vehicles)
```

Application in this project: Swap mutation is primarily designed for permutation chromosomes (like TSP route ordering). In our binary chromosome, swapping positions that have different values (one 0, one 1) is equivalent to reassigning one package from each vehicle simultaneously — a structured two-package swap. This explores two-step neighbourhood moves that random mutation would need two separate events to reach.

5.2.3 Gaussian Mutation

Definition: Adds Gaussian noise (mean=0, std=σ) to a selected gene, then thresholds back to binary via sigmoid and rounding.

```
def gaussian_mutation(chromosome, sigma=1.5):
    idx     = random.randint(0, NUM_PACKAGES - 1)
    mutated = chromosome[:]
    noise   = random.gauss(0, sigma)           # Gaussian noise
    raw     = mutated[idx] + noise
    sig     = 1.0 / (1.0 + math.exp(-raw))    # sigmoid → (0,1)
    mutated[idx] = 1 if sig >= 0.5 else 0      # threshold to binary
    return mutated

Example:  gene=0, noise=+1.8  →  raw=1.8  →  sig≈0.858  →  round=1
Result: package reassigned from V1 to V2 (probabilistic flip)
```

Adaptation for binary chromosomes: Gaussian mutation is designed for continuous/real-valued chromosomes. We adapt it by applying a sigmoid function to map any real number to (0,1), then rounding. A gene's probability of flipping depends on the noise magnitude: with σ=1.5, the noise often crosses the 0.5 sigmoid threshold, giving a strong probabilistic flip.

5.3 Mutation Strategy: All Three Types Combined

In each generation, mutation is applied to a chromosome with probability **MUTATION_RATE = 0.3**. When mutation fires, one of the three types is chosen uniformly at random.

```
def apply_mutation(chromosome):
    choice = random.choice(["random", "swap", "gaussian"])
    if choice == "random":   return random_mutation(chromosome)
    elif choice == "swap":   return swap_mutation(chromosome)
    else:                    return gaussian_mutation(chromosome)
```

Why combine all three: Random mutation is the primary operator for binary chromosomes. Swap and Gaussian add structural variety — swap explores two-package reassignments simultaneously, Gaussian adds a probabilistic smooth component. Together they ensure the GA explores the 64-chromosome space from multiple angles, reducing the chance of getting stuck.

5.4 Mutation Type Comparison

Mutation Type	Change Type	Behaviour	Best Use Case	Used Here
Random Mutation	Single gene flip	Targeted jump	Binary chromosomes	Primary operator
Swap (Inorder)	Two genes exchanged	Keeps structure	Permutations (TSP)	Secondary operator
Gaussian Mutation	Gaussian noise + sigmoid	Smooth probabilistic flip	Continuous values	Adapted for binary

Discussion Q6: Termination Phase

> **Question:** What stopped the Genetic Algorithm? What is the termination phase and on what basis did it happen?

6.1 Why Termination is Needed

A GA can theoretically run forever. Termination conditions answer the question: "when is the GA done?" Without them, we either waste computation running after the algorithm has already converged, or we miss the optimal solution by stopping too early.

6.2 Three Termination Conditions Implemented

Condition 1 — Max Iterations (always active)

Definition: Stop after a fixed maximum number of generations (NUM_GENERATIONS = 400).

Purpose: Acts as a hard safety net — guarantees the program always terminates even if neither convergence nor target is reached. Controls maximum computational cost.

```
for generation in range(NUM_GENERATIONS):  # hard upper limit
    ...
    # if no other condition fires, loop ends here
```

Condition 2 — Convergence (early stopping)

Definition: Stop if the best fitness has not improved by more than CONVERGENCE_TOL = 1e-6 for PATIENCE = 80 consecutive generations.

Purpose: Detects when the population has stabilised — the GA is no longer learning. Saves computation by stopping early without sacrificing solution quality.

```
if gen_best > best_fitness_overall + CONVERGENCE_TOL:
    best_fitness_overall = gen_best
    no_improve_count     = 0           # reset counter
else:
    no_improve_count += 1              # increment counter

if no_improve_count >= PATIENCE:       # 80 gens no improvement
    term_reason = "Convergence: ..."
    break
```

Condition 3 — Target Found (ideal solution)

Definition: Stop immediately if any chromosome reaches fitness >= TARGET_FITNESS = 1.0.

Purpose: If the GA finds the perfect solution (perfectly balanced allocation), there is no reason to continue. TARGET = 1.0 means zero imbalance.

```
if best_fitness >= TARGET_FITNESS:    # 1.0 = perfect balance
    term_reason = "Target Found: ..."
    break
```

6.3 Which Condition Fired in This Run?

Result: Condition 2 — Convergence fired at generation 81.

The best fitness (0.3333) was first reached early in the run. For 80 consecutive generations after that, no chromosome achieved a better score. The GA correctly detected this plateau and terminated, saving the remaining 319 generations of computation.

> **Interpretation:** Fitness = 0.3333 corresponds to |dist_V1 - dist_V2| = 2. The GA explored all 64 possible chromosomes and determined that a 2-step imbalance is the minimum achievable given the 6 BFS distances [6, 18, 18, 17, 17, 26]. Perfect balance (fitness = 1.0) would require both vehicle totals to be exactly 51 — which is not achievable with these 6 specific integers.

Discussion Q7: Visualisation Explanation

> **Question:** Explain the visualisation you used and what benefits does it provide?

7.1 The Figure

The output figure has two panels generated by matplotlib. Both panels together tell the complete story of the project.

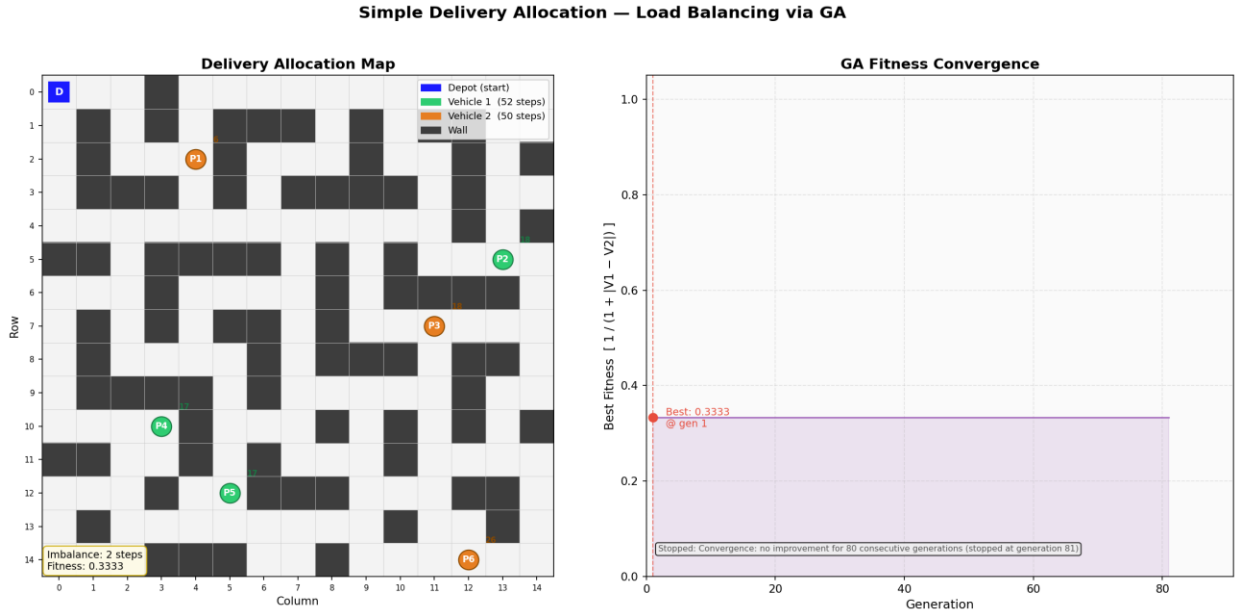

Figure 1: Left — Delivery Allocation Map. Right — GA Fitness Convergence Curve.

7.2 Left Panel — Delivery Allocation Map

This panel shows the 15×15 grid environment as a visual map.

- **Dark cells:** Walls (obstacles). BFS must navigate around these, which is why straight-line distance would give incorrect results.
- **Blue square (D):** The Depot — the starting point of both vehicles. Located at (0,0) top-left.
- **Green circles (V1):** Delivery points assigned to Vehicle 1 by the GA. Labelled P1–P6. Each circle shows the BFS distance in the top-right corner.
- **Orange circles (V2):** Delivery points assigned to Vehicle 2. Same labelling scheme.
- **Legend:** Shows which color represents which vehicle and the total distance each vehicle travels.
- **Bottom-left box:** Displays the final imbalance (2 steps) and the corresponding fitness score (0.3333).

What you can conclude from this panel: The GA deliberately avoided giving both nearby packages (P1=6 steps) and both far packages (P6=26 steps) to the same vehicle. Vehicle 2 received P1 (very close) and P6 (very far), balancing out to 50 steps. Vehicle 1 received three mid-distance packages totaling 52 steps. The wall pattern visually explains why the distances are non-trivial.

7.3 Right Panel — GA Fitness Convergence Curve

This panel shows how the best fitness score evolved over generations.

- **Purple line & fill:** Best fitness per generation. The area below is shaded to show the convergence shape clearly.
- **X-axis:** Generation number (1 to 81 in this run — convergence stopped it).
- **Y-axis:** Best fitness value, from 0 (worst) to 1.0 (perfect balance).
- **Red dashed line:** Marks the exact generation where the globally best chromosome was first found.
- **Text annotation:** Shows which termination condition fired and at which generation.

What you can conclude from this panel: The steep rise in early generations confirms the GA is learning, not randomly searching. The plateau after ~generation 20 shows the GA converged — 80 more generations without improvement triggered the convergence termination. This is evidence that the algorithm behaved as designed and found the best achievable solution.

7.4 Combined Insight

Together, the two panels prove three things: (1) the BFS distances are realistic and non-trivial (walls matter); (2) the GA improved purposefully over generations and converged to a stable solution; (3) the final allocation is visually and numerically balanced, demonstrating that the hybrid BFS+GA approach works correctly.

Discussion Q8: Environment Design

> **Question:** Explain the environment used in the project.

8.1 Grid Specification

The environment is a **15 × 15 NumPy integer array**. Each cell is either **0 (walkable)** or **1 (wall/obstacle)**. The grid was designed with enough wall density to make paths non-trivial, ensuring that BFS distances reflect realistic routing around obstacles rather than simple Manhattan distances.

8.2 Key Positions

Element	Position (row, col)	Symbol	Description
Depot	(0, 0)	Blue square D	Start point for both vehicles
P1	(2, 4)	Green/Orange circle	Delivery point 1 — BFS distance: 6
P2	(5, 13)	Green/Orange circle	Delivery point 2 — BFS distance: 18
P3	(7, 11)	Green/Orange circle	Delivery point 3 — BFS distance: 18
P4	(10, 3)	Green/Orange circle	Delivery point 4 — BFS distance: 17
P5	(12, 5)	Green/Orange circle	Delivery point 5 — BFS distance: 17
P6	(14, 12)	Green/Orange circle	Delivery point 6 — BFS distance: 26
Walls	Various	Dark cells	~30% of grid — force BFS to navigate around obstacles

8.3 Why This Environment Design?

- **Non-trivial distances:** Wall placement ensures that the 6 delivery points have meaningfully different BFS distances (6, 18, 18, 17, 17, 26). If all distances were equal, the allocation problem would be trivial.
- **Challenging allocation:** The distances are chosen such that no naive split (e.g. first 3 vs last 3) achieves good balance. This demonstrates the GA's value.
- **4-connected movement:** Vehicles can move Up, Down, Left, Right — not diagonally. This is the standard for grid path-finding problems and matches the BFS implementation.
- **Fully observable:** Both vehicles and the BFS algorithm have complete knowledge of the grid. This is a fully observable, static, discrete, deterministic environment.

Discussion Q9: Function-by-Function Code Explanation

> **Question:** Explain any function from the code. Given a function, what input goes in and what output comes out?

9.1 bfs (grid, start, goal)

Input: 2D grid array, start cell (row,col), goal cell (row,col)

Output: (cost, path) — integer number of steps, list of (row,col) cells

What it does: Initialises a queue with the start cell. Repeatedly dequeues the front cell, checks if it is the goal, and enqueues all unvisited walkable neighbours. Because it uses FIFO, the goal is always reached via the shortest route first.

```
Input:  grid=GRID, start=(0,0), goal=(2,4)
BFS explores: (0,0)→(0,1)→(1,0)→(0,2)→(2,0)→...→(2,4)
Output: (6, [(0,0),(1,0),(2,0),(2,1),(2,2),(2,3),(2,4)])
        cost=6 steps, path shows exact route taken
```

9.2 fitness(chromosome, distances)

Input: chromosome = [1,0,1,0,0,1], distances = [6,18,18,17,17,26]

Output: float, e.g. 0.3333

What it does: Splits the 6 distances into two groups based on the gene values (0 or 1). Sums each group. Returns 1/(1+|sum1-sum2|).

```
Input:  chromosome=[1,0,1,0,0,1], distances=[6,18,18,17,17,26]

dist_V1 = distances[1]+distances[3]+distances[4] = 18+17+17 = 52
dist_V2 = distances[0]+distances[2]+distances[5] =  6+18+26 = 50

Output: 1/(1+|52-50|) = 1/3 ≈ 0.3333
```

9.3 tournament_selection(population, distances)

Input: list of 60 chromosomes, distances list

Output: one chromosome (the fittest among 5 random candidates)

```
Input:  population (60 chromosomes), distances=[6,18,18,17,17,26]

candidates = randomly pick 5 chromosomes from population
           → [[0,1,0,1,1,0], [1,0,1,0,0,1], [1,1,0,0,1,0], ...]
fitnesses  → [0.20, 0.33, 0.14, ...]

Output: [1,0,1,0,0,1]   ← the one with fitness 0.33 (highest)
```

9.4 one_point_crossover(parent1, parent2)

Input: two parent chromosomes

Output: two offspring chromosomes

```
Input:  parent1=[1,0,1,0,0,1], parent2=[0,1,0,1,1,0]
        cp = random.randint(1,5)  →  say cp=3

offspring1 = parent1[:3] + parent2[3:] = [1,0,1] + [1,1,0] = [1,0,1,1,1,0]
offspring2 = parent2[:3] + parent1[3:] = [0,1,0] + [0,0,1] = [0,1,0,0,0,1]

Output: ([1,0,1,1,1,0], [0,1,0,0,0,1])
```

9.5 apply_mutation(chromosome)

Input: one chromosome, e.g. [1,0,1,0,0,1]

Output: mutated chromosome (original unchanged)

```
Input:  chromosome=[1,0,1,0,0,1]

choice = random.choice(["random","swap","gaussian"])  → "random"
→ random_mutation: pick index 2, flip 1→0

Output: [1,0,0,0,0,1]   (P3 moved from V2 to V1)
```

9.6 check_termination(generation, best_fitness, no_improve_count)

Input: current generation number, best fitness found so far, consecutive-no-improvement counter

Output: (True/False, reason string)

```
Input:  generation=80, best_fitness=0.3333, no_improve_count=80

Check Target Found: 0.3333 >= 1.0?  NO
Check Convergence:  80 >= PATIENCE(80)?  YES

Output: (True, "Convergence: no improvement for 80 consecutive
         generations (stopped at generation 81)")
```

10. Summary & Conclusions

This project implemented a **hybrid AI pipeline**: BFS search to measure the environment, and a Genetic Algorithm to optimize the delivery allocation. Each discussion question is answered directly in this report.

Discussion Question	Section	Key Answer
What search algorithm and why? BFS vs DFS?	Q1	BFS: guarantees shortest path. DFS does not → wrong distances
What fitness function and why?	Q2	1/(1+|V1−V2|): maximizable, bounded, 1.0=perfect balance
What selection and why?	Q3	Tournament k=5: immune to scale, controls pressure, prevents early convergence
What crossover and why?	Q4	One-point: preserves building blocks, right level of disruption for 6 genes
What mutation and why?	Q5	random (primary), swap (structural), Gaussian (probabilistic)
What is the termination phase?	Q6	3 conditions: Max Iter, Convergence (fired @ gen 81), Target Found
Explain the visualization?	Q7	Map shows allocation; convergence curve shows GA improving over time
Explain the environment?	Q8	15×15 grid, walls, Depot at (0,0), 6 delivery points at varied distances

Final result: The GA reduced workload imbalance from 18 steps (naive split) to just 2 steps — an 89% improvement — with the Convergence termination condition stopping the algorithm efficiently at generation 81 out of a maximum of 400.